

Unit 3: Deadlock

System Model

- A system consists of a finite number of resources to be distributed among a number of competing processes. The resources may be partitioned into several types (or classes), each consisting of some number of identical instances.
- CPU cycles, files, and I/O devices (such as printers and DVD drives) are examples of resource types. If a system has two CPUs, then the resource type CPU has two instances. Similarly, the resource type printer may have five instances.
- A process must request a resource before using it and must release the resource after using it. A process may request as many resources as it requires carrying out its designated task.
- Obviously, the number of resources requested may not exceed the total number of resources available in the system. In other words, a process cannot request three printers if the system has only two.
- Under the normal mode of operation, a process may utilize a resource in only the following sequence:
 1. **Request.** The process requests the resource. If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.
 2. **Use.** The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer).
 3. **Release.** The process releases the resource.
- A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set.
- The events with which we are mainly concerned here are resource acquisition and release. The resources may be either physical resources (for example, printers, tape drives, memory space, and CPU cycles) or logical resources (for example, semaphores, mutex locks, and files).

Deadlock Characterization

In a deadlock, processes never finish executing, and system resources are tied up, preventing other jobs from starting.

Necessary Conditions

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

1. Mutual exclusion. At least one resource must be held in a nonsharable mode; that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
2. Hold and wait. A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
3. No preemption. Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
4. Circular wait. A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , ..., P_{n-1} is waiting for a resource held by P_n , and P_n is waiting for a resource held by P_0 .

We emphasize that all four conditions must hold for a deadlock to occur. The circular-wait condition implies the hold-and-wait condition, so the four conditions are not completely independent.

Resource Allocation Graph

- Deadlocks can be described more precisely in terms of a directed graph called a system resource-allocation graph. This graph consists of a set of vertices V and a set of edges E .
- The set of vertices V is partitioned into two different types of nodes: $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the active processes in the system, and $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.
- A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$; it signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.
- A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$; it signifies that an instance of resource type R_j has been allocated to process P_i .
- A directed edge $P_i \rightarrow R_j$ is called a request edge; a directed edge $R_j \rightarrow P_i$ is called an assignment edge.
- Pictorially, we represent each process P_i as a circle and each resource type R_j as a rectangle. Since resource type R_j may have more than one instance, we represent each such instance as a dot within the rectangle. Note that a request edge points to only the rectangle R_j , whereas an assignment edge must also designate one of the dots in the rectangle.
- When process P_i requests an instance of resource type R_j , a request edge is inserted in the resource-allocation graph. When this request can be fulfilled, the request edge is instantaneously transformed to an assignment edge. When the process no longer needs access to the resource, it releases the resource. As a result, the assignment edge is deleted.

- The resource-allocation graph in the below figure depicts the following situation.
- The sets P, R, and E:
 - $P = \{P1, P2, P3\}$
 - $R = \{R1, R2, R3, R4\}$
 - $E = \{P1 \rightarrow R1, P2 \rightarrow R3, R1 \rightarrow P2, R2 \rightarrow P2, R2 \rightarrow P1, R3 \rightarrow P3\}$
- Resource instances:
 - One instance of resource type R1
 - Two instances of resource type R2
 - One instance of resource type R3
 - Three instances of resource type R4
- Process states:
 - Process P1 is holding an instance of resource type R2 and is waiting for an instance of resource type R1.
 - Process P2 is holding an instance of R1 and an instance of R2 and is waiting for an instance of R3.
 - Process P3 is holding an instance of R3.
- Given the definition of a resource-allocation graph, it can be shown that, if the graph contains no cycles, then no process in the system is deadlocked. If the graph does contain a cycle, then a deadlock may exist.
- If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred. If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred. Each process involved in the cycle is deadlocked. In this case, a cycle in the graph is both a necessary and a sufficient condition for the existence of deadlock.
- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

Methods for Handling Deadlocks

- Generally speaking, we can deal with the deadlock problem in one of three ways:
 - We can use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlocked state.
 - We can allow the system to enter a deadlocked state, detect it, and recover.
 - We can ignore the problem altogether and pretend that deadlocks never occur in the system.
- The third solution is the one used by most operating systems, including Linux and Windows. It is then up to the application developer to write programs that handle deadlocks.

- To ensure that deadlocks never occur, the system can use either deadlock prevention or a deadlock-avoidance scheme. Deadlock prevention provides a set of methods to ensure that at least one of the necessary conditions cannot hold. These methods prevent deadlocks by constraining how requests for resources can be made.
- Deadlock avoidance requires that the operating system be given additional information in advance concerning which resources a process will request and use during its lifetime. With this additional knowledge, the operating system can decide for each request whether or not the process should wait.
- To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.
- If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may arise. In this environment, the system can provide an algorithm that examines the state of the system to determine whether a deadlock has occurred and an algorithm to recover from the deadlock (if a deadlock has indeed occurred).

Deadlock Prevention

For a deadlock to occur, each of the four necessary conditions must hold. By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock.

1. Mutual Exclusion

- The mutual exclusion condition must hold. That is, at least one resource must be nonsharable. Sharable resources, in contrast, do not require mutually exclusive access and thus cannot be involved in a deadlock. Read-only files are a good example of a sharable resource.
- If several processes attempt to open a read-only file at the same time, they can be granted simultaneous access to the file. A process never needs to wait for a sharable resource.
- In general, however, we cannot prevent deadlocks by denying the mutual-exclusion condition, because some resources are intrinsically nonsharable.

2. Hold and Wait

- To ensure that the hold-and-wait condition never occurs in the system, we must guarantee that, whenever a process requests a resource, it does not hold any other resources.
- One protocol that we can use requires each process to request and be allocated all its resources before it begins execution.
- An alternative protocol allows a process to request resources only when it has none. A process may request some resources and use them. Before it can request any additional resources, it must release all the resources that it is currently allocated.

- Both these protocols have two main disadvantages. First, resource utilization may be low, since resources may be allocated but unused for a long period.
- Second, starvation is possible. A process that needs several popular resources may have to wait indefinitely, because at least one of the resources that it needs is always allocated to some other process.

3. No Preemption

- The third necessary condition for deadlocks is that there be no preemption of resources that have already been allocated. To ensure that this condition does not hold, we can use the following protocol.
 - If a process is holding some resources and requests another resource that cannot be immediately allocated to it (that is, the process must wait), then all resources the process is currently holding are preempted. In other words, these resources are implicitly released. The preempted resources are added to the list of resources for which the process is waiting. The process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.
 - Alternatively, if a process requests some resources, we first check whether they are available. If they are, we allocate them. If they are not, we check whether they are allocated to some other process that is waiting for additional resources. If so, we preempt the desired resources from the waiting process and allocate them to the requesting process. If the resources are neither available nor held by a waiting process, the requesting process must wait.

4. Circular Wait

- The fourth and final condition for deadlocks is the circular-wait condition. One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.
- On contrast, so as to demand resource R_j , a process should initially discharge all R_i with the end goal that $i \geq j$.
- One major test in this plan is deciding the overall requesting of the various resources.

Approach

- A natural number is allotted to each resource.
- Each process is permitted to demand for the resources either in just ascending or descending order of the resource number.
- In the event that ascending request is followed, in the event that a process requires a lesser number resource, at that point it must release every one of the resources having bigger number and the other way around.
- This methodology is the most functional methodology and implementable.

- Nonetheless, this methodology may cause starvation however will never prompt deadlock.

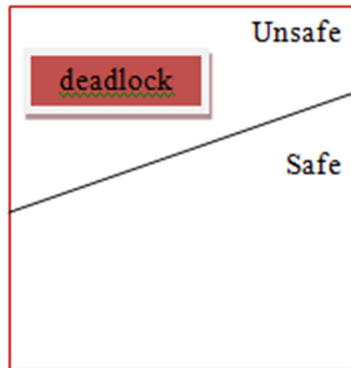
Deadlock Avoidance

- Deadlock-prevention algorithms prevent deadlocks by limiting how requests can be made. The limits ensure that at least one of the necessary conditions for deadlock cannot occur. Possible side effects of preventing deadlocks by this method, however, are low device utilization and reduced system throughput.
- An alternative method for avoiding deadlocks is to require additional information about how resources are to be requested.
- With this knowledge of the complete sequence of requests and releases for each process, the system can decide for each request whether or not the process should wait in order to avoid a possible future deadlock.
- Each request requires that in making this decision the system consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.
- The various algorithms that use this approach differ in the amount and type of information required. The simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need.
- Given this a priori information, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state. A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular-wait condition can never exist.
- The resource allocation state is defined by the number of available and allocated resources and the maximum demands of the processes.

Safe State

- A state is safe if the system can assign resources to each process (up to its most extreme) in some sequence and still will be able maintain a deadlock free state.
- In other words, a system is in a safe state if there exists a safe sequence.
- An arrangement of processes $\langle P_1, P_2, \dots, P_n \rangle$ is a safe arrangement for the present allocation state if, for each P_i , the resource requests that P_i make can be fulfilled by the currently accessible resources in addition to the resources held by all P_j , with $j < i$.
- In this circumstance, the resources that P_i needs are not quickly available, at that point then P_i can wait until all P_j have completed.
- When all P_j have completed, P_i can acquire the majority of its required resources, complete its assigned task, and return its allotted resources, and will terminate.
- At the point when P_i ends, P_{i+1} can get its required resources, and so on.

- In the event that no such sequence exists, at that point the system state is said to be unsafe.
- A safe state is definitely not a deadlock state. On the other hand, a deadlock state is a unsafe state. All the unsafe state are not deadlock.



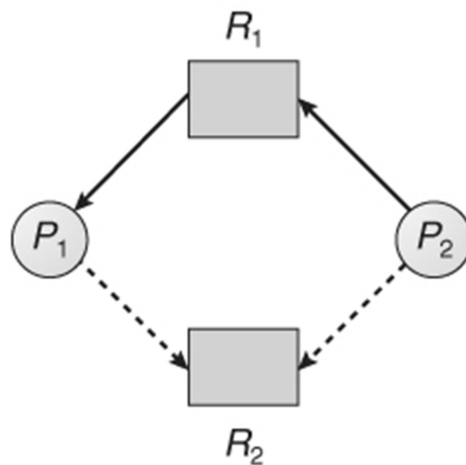
- Consider a system with 10 magnetic tape drives and 3 processes P0, P1, and P2 respectively. The maximum requirement and current needs by each process is shown in table below.

Process	Maximum needs	Current allocated
P0	8	4
P1	4	2
P2	8	2

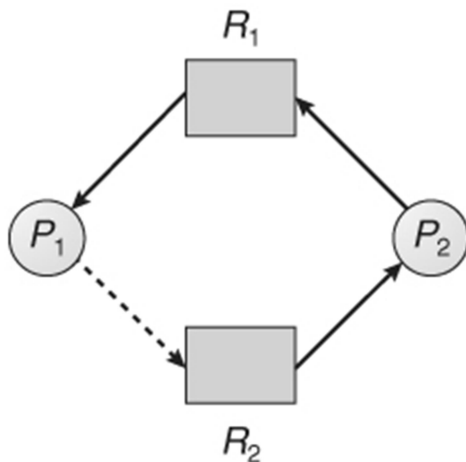
- Process P0 is having 4 tape drives, P1 is having 2 tape drives and P2 is having 2 tape drives.
- For a safe sequence process P0 needs 4 more tape drives to complete its execution. Similarly process P1 needs 2 and P2 needs 6. Total available resources are 2.
- Since process P0 needs 5 tape drives, but available is only 2, so process P1 will wait till it gets resources. Process P1 needs only 2 so its request will be fulfilled and as P1 will finish its execution it will return all resources.
- Now currently available resources are 4. Then process P2 will wait as available resources are 4 and P2 needs 6.
- The system will now fulfill the needs of P0. Process P0 will execute and release its all resources. Now available resources are 8. And in the last request of process P2 will be fulfilled.
- So, the safe sequence will be <P1, P0, P2>.
- There is one thing to be noted here that any system can go from a safe state to an unsafe state if any process requests more resources.

Resource-Allocation-Graph Algorithm

- Deadlock can be detected in a system through cycles only if the resources have just single instances in the resource-allocation graphs.
- For this situation, unsafe states can be identified and avoided by adding a claim edges in the resource-allocation graph, represented by dashed lines, which point from a process to a resource that it might request later on.
- All claim edges must be added to the graph for a specific process before that process is allowed to demand any resources.
- At the point when a process makes a request, the claim edge $P_i \rightarrow R_j$ is changed over to a request edge. Essentially when a resource is released, the task returns to a claim edge.
- This methodology works by denying demands that would deliver cycles in the resource-allocation graph, producing claim edges into results.
- Consider for instance what happens when process P_2 request resource R_2 :



- The subsequent resource-allocation graph would have a cycle in it, thus the request can't be granted.



Banker's Algorithm

- Resource allocation graph works only for single instance. In case of more than one instance an alternative approach is used known as bankers algorithm.
- The Banker's Algorithm gets its name since it is a strategy that bankers could use to guarantee that when they loan out resources they will even now have the option to fulfill every one of their clients request.
- At the point when a process starts, it must state ahead of time the maximum resources it might ask for, up to the amount available on the system.
- At the point when a request is made, the scheduler decides if giving the request would leave the system in a safe state. In case, the process did not get a chance it must wait until the request can be granted securely.
- The banker's algorithm depends on a few key data structures: (where n is the quantity of processes and m is the quantity of resource types.)
 - Available[m] demonstrates what numbers of resources are at present available of each kind.
 - Max[n][m] demonstrates the maximum request of each process of every resource.
 - Allocation[n][m] demonstrates the quantity of every resource class allocated to each process.
 - Need[n][m] demonstrates the rest of the resources required of each type for each process. (Note that $\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$ for all i, j)
- For simplification of concept, we mention the accompanying notations/objective facts:
 - One row of the Need vector, Need[i], can be treated as a vector comparing to the requirements of process i, and comparatively for Allocation and Max.
 - A vector X is viewed as $\leq$ a vector Y if $X[i] \leq Y[i]$ for all i.

Safety Algorithm

We would now be able to exhibit the algorithm for seeing if or not a system is in a safe state. This algorithm can be portrayed as follows:

1. Assume Work and Finish to be vectors of length m and n, individually. Initialize
Work = Available and Finish[i] = false for $i = 0, 1, \dots, n - 1$.
2. Find an index i with the end goal that both
 - a. Finish[i] == false
 - b. $\text{Need}_i \leq \text{Work}$On the off chance that no such i exists, go to step 4.
3. Work = Work + Allocation;
Finish[i] = true
Go to step 2.
4. In case Finish[i] == true for all i, at that point the system is in a safe state.

This algorithm may require a request for $m \times n^2$ operations to decide if a state is safe or not.

Resource-Request Algorithm

- Next, we depict the algorithm for deciding if requests can be securely allowed or not.
- Assume $Request_i$ be the request vector for process P_i . In the event that $Request_i[j] = k$, at that point process P_i needs k instances of resource type R_j . At the point when a request for resources is made by process P_i , the accompanying actions are made:
 1. If $Request_i \leq Need_i$, go to step 2. Otherwise, raise an error condition, since the process has exceeded its maximum claim.
 2. In the event that $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since the resources are not accessible.
 3. Have the system claim to have assigned the requested resources to process P_i by modifying the state as follows:

$$Available = Available - Request_i ;$$

$$Allocation_i = Allocation_i + Request_i ;$$

$$Need_i = Need_i - Request_i ;$$

4. In case the final resource-allocation state is safe, the transaction is finished, and process P_i is allocated its resources. Nonetheless, if the new state is unsafe, at that point P_i must wait for $Request_i$, and the old resource-allocation state is restored.

Example

Considering a system with five processes P_0 through P_4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t_0 following snapshot of the system has been taken:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Question1. What will be the content of the Need matrix?

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$

So, the content of Need Matrix is:

Process	Need		
	A	B	C

P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Question2. Is the system in a safe state? If yes, then what is the safe sequence?

Applying the Safety algorithm on the given system,

For P0, Need [7 4 3] < Available [3 3 2]

Finish = False, P0 will wait.

For P1, Need [1 2 2] < Available [3 3 2]

Finish = True

Available = Available + Allocation = [3 3 2] + [2 0 0] = [5 3 2]

For P2, Need [6 0 0] = Available [5 3 2]

Finish = False, P2 will wait.

For P3, Need [0 1 1] = Available [5 3 2]

Finish = True

Available = Available + Allocation = [5 3 2] + [2 1 1] = [7 4 3]

For P4, Need [4 3 1] = Available [7 4 3]

Finish = True

Available = Available + Allocation = [7 4 3] + [0 0 2] = [7 4 5]

For P0, Need [7 4 3] = Available [7 4 5]

Finish = True

Available = Available + Allocation = [7 4 5] + [0 1 0] = [7 5 5]

For P2, Need [6 0 0] = Available [7 5 5]

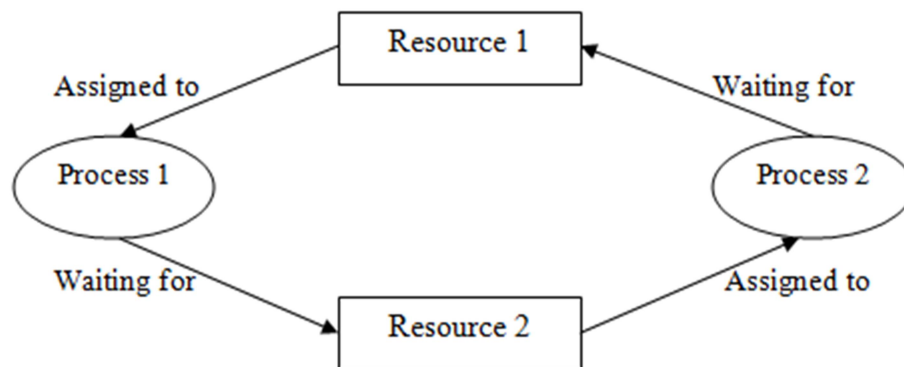
Finish = True

Available = Available + Allocation = [7 5 5] + [3 0 2] = [10 5 7]

Therefore, the safe sequence is <P1, P3, P4, P0, P2>

Deadlock Detection

- Deadlock detection is the process of really verifying that a deadlock exists and distinguishing the processes and resources engaged in the deadlock.
- The essential thought is to check allocate against resource availability for all conceivable allocation sequences to decide whether the system is in deadlocked state.
- Obviously, the deadlock detection algorithm is just 50% of this procedure. When a deadlock is recognized, there should be an approach to recover a few choices exists:
 1. **Single instance of resource:** Here to detect deadlock, we can run an algorithm to check for cycle in the Resource Allocation Graph. Presence of cycle in the graph is the adequate condition for deadlock.



In the above graph, resource 1 and resource 2 have single instances. There is a cycle $R1 \rightarrow P1 \rightarrow R2 \rightarrow P2$. Thus, Deadlock is confirmed.

2. **Multiple instance of resources:** Detection of the cycle is essential yet not an adequate condition for deadlock detection, for this situation, the system might be in a halt fluctuates as per various circumstances.

Recovery from Deadlock

At the point when a Deadlock Detection Algorithm verifies that a deadlock has happened in the system, the system must recover from that deadlock. There are two methodologies of breaking a Deadlock:

1. **Process Termination:** To wipe out the from the deadlock, we can just kill at least one processes. For this, we utilize two techniques:
 - a. **Abort all the Deadlocked Processes:** Aborting every one of the processes will absolutely break the deadlock, however with an incredible costs. The deadlocked processes may have computed for quite a while and the consequence of those incomplete calculations must be disposed of and there is a likelihood to recalculate them later.
 - b. **Abort one process in turn until deadlock is removed:** Abort one deadlocked process in turn, until deadlock cycle is removed from the system. Because of this

technique, there might be reasonable overhead, in light of the fact that subsequent to aborting of process, we need to run deadlock detection algorithm to check whether any processes are yet deadlocked.

2. **Resource Preemption:** To remove deadlock we can utilize resource pre-emption. We preempt a few resources from processes and give those resources to different processes.

This strategy will raise three issues –

- a. **Choosing a victim:** We should figure out which resources and which processes are to be preempted and furthermore the request to limit the expense.
- b. **Rollback:** We should figure out what action will be taken for the process from which resources are preempted. One solution is all rollback. That implies aborting the process and restarts it.
- c. **Starvation:** In a system, it might happen that equivalent process is constantly picked as a victim. Subsequently, that process will never finish its assigned task. This circumstance is called Starvation and must be kept away from. One arrangement is that a process must be picked as a victim just a limited number of times.